第 2-1 讲 • 主角登场：主进程与渲染进程

> **模块 2 • 认清记账簿的骨架** ｜ 建议用时 **40 分钟**（阅读 22 + 动手 18）
> 前置要求：完成模块 0、1

本节目标

读完并做完这一讲，你能：

1. 说出记账簿里**两个主角**分别是谁、各自负责什么，并能背出那句口诀。
2. 解释**为什么界面不能直接读数据库**——不是"这样做不好"，而是"根本做不到"。
3. 在代码里指出**安全配置的三行**（`contextIsolation` / `nodeIntegration` / `webSecurity`），并说明它们在防什么。
4. 用一条命令**证明**：渲染层碰不到文件系统，主进程到处都是。
5. 理解这个架构的好处与代价（为什么要多写一层代码）。

一、两个主角

1.1 现实比喻：餐厅

这是整套讲义最重要的一组比喻，请务必记牢：

角色	对应	能做什么	不能做什么
后厨	**主进程**（main）	碰数据库、读写文件、调系统、开窗口	不直接管界面好不好看
服务员	**渲染进程**（renderer）	画界面、响应用户点击、显示数据	**碰不到冷库（数据库）、碰不到系统**
传菜口	**preload**	后厨和服务员之间**唯一的**合法通道	不能绕过规则
点菜单	**IPC**	服务员往后厨递的那张纸	必须写清楚要什么

口诀：后厨管数据，服务员管笑脸，中间只走传菜口。

1.2 为什么必须分成两个人

你可能会想：一个人干完不就好了吗？为什么搞得这么复杂？

三个真实原因：

① 安全：界面代码是"不可信"的

界面要显示用户输入、要加载各种内容。如果它有权限读写文件，那么**一个恶意网页、一段被注入的脚本，就能删掉你的账本。**

分进程之后，界面被关进"玻璃房"：**它连"文件"是什么都看不到**，只能通过传菜口说"我要保存一笔账"，由后厨去执行。

② 稳定：界面崩了不连累数据

你在第 1-1 讲亲眼看过了：结束渲染进程，窗口变白，但**主进程和你的数据都还在。**如果是一个进程，界面崩溃就可能带着正在写的数据一起完蛋。

③ 分工：各用各的技术

主进程跑在 Node 环境（会读写文件、连数据库），渲染进程跑在浏览器环境（会画界面、做动画）。**两边的能力和限制不一样**，硬塞在一起会互相打架。

1.3 这个架构的代价（要诚实）

坏处：写一个功能要动好几个文件。

比如"界面上一按按钮就保存一笔账"，需要：

```
界面按钮（渲染进程写）→ 传菜口（preload）→ 点菜单（IPC 名字）→ 后厨执行（主进程写数据库）
```

四个地方都要碰。这也是为什么你以后改功能时，我会提醒你"这一改要动几个文件"。

但要记住：这不是记账簿设计得啰嗦，而是**所有正经桌面应用都这么干**（VS Code、微信桌面版、Discord全是这个架构）。**这份"啰嗦"买来的是安全和稳定。**

二、代码里长什么样

2.1 项目里的三个目录，正好对应三个角色

```
src\
├─ main\        ← 后厨（主进程）：管数据、系统、窗口
├─ preload\     ← 传菜口：唯一的合法通道
├─ renderer\    ← 服务员（渲染进程）：只管界面
└─ shared\      ← 共用：类型定义、常量、纯计算（两边都要用）
```

`shared` **的存在说明一件事**：有些东西两边都要知道，比如"一笔账有哪些字段"。总不能后厨说"金额"，服务员理解成"备注"。所以把这类**共同约定**放在 `shared\`，两边都引用它，保证口径一致。

2.2 真凭实据：谁能碰系统

下面是**我从项目里实际统计出来的**（你可以自己验证）：

检查项	结果
主进程里导入系统模块（`node:fs` 读写文件、`node:crypto` 加密、`node:http` 网络……）	**12 处以上**
渲染层里导入系统模块	**0 处**

渲染层一处都没有。 这不是巧合，是**架构强制**的结果——渲染进程根本拿不到那些能力。

2.3 安全配置：三行代码定生死

打开 `src\main\window.ts`，你会看到创建窗口时的关键配置：

```
webPreferences: {
  preload: path.join(__dirname, '../preload/index.js'),
  contextIsolation: true,      // ① 隔离：界面代码和 preload 各用各的环境
  nodeIntegration: false,      // ② 禁用：界面里不能直接用 Node 的能力
  webSecurity: true,           // ③ 开启：浏览器安全策略照常生效
  sandbox: false               // ④ 沙箱（本项目关掉了，见下）
}
```

逐条翻译成人话：

配置	意思	如果写反了会怎样
`contextIsolation: true`	界面代码和通道代码**互相看不见对方的变量**	界面代码能篡改通道，等于传菜口被砸开
`nodeIntegration: false`	界面里**不能**写 `require('fs')` 去读文件	界面直接能删你的账本
`webSecurity: true`	浏览器的同源策略等安全规则**照常生效**	界面能随意加载外部脚本
`sandbox: false`	关掉了最严格的沙箱（本项目需要它来加载本地模型文件）	开着更安全，但会限制部分本地能力

这三行（前三条）是 Electron 的安全底线。 你在任何 Electron 项目里看到 `nodeIntegration: true`，基本可以判定这个项目有严重安全隐患。

三、动手任务

步骤 1：亲手验证"渲染层碰不到系统"

这是本讲最重要的动手。执行：

```
cd "C:\Users\34123\Desktop\记账簿"

# 渲染层有没有导入系统模块？
Select-String -Path src\renderer\src\**\*.ts,src\renderer\src\**\*.tsx -Pattern "from 'nod

# 主进程有没有导入系统模块？
Select-String -Path src\main\**\*.ts -Pattern "from 'node:" -ErrorAction SilentlyContinue
```

预期结果：第一个是 **0**，第二个是**十几到几十。**

这个数字对比就是架构的证据：

- 渲染层是 **0** → 它拿不到文件系统、进程、加密等能力 → **它想干坏事也没工具**
- 主进程有很多 → 它才是真正"能碰电脑"的那个 → 所以它必须谨慎

步骤 2：找出安全配置

```
Get-Content src\main\window.ts | Select-String "contextIsolation|nodeIntegration|webSecuri
```

预期看到四行，就是上面讲的那四条。

思考：如果我把 `nodeIntegration` 改成 `true`，会发生什么？

答案：界面代码就能直接读写你的文件了——**包括你的账本。**这就是为什么这个值必须是 `false`，而且没有任何正当理由去改它。

步骤 3：找出主进程的入口顺序（理解启动流程）

```
Select-String -Path src\main\index.ts -Pattern "registerAllIpc|initDatabase|createMainWind
```

预期看到类似：

```
36: registerAllIpc()        ← 先装好"点菜单"通道
41: const result = initDatabase(config.dataDir)   ← 再打开数据库
172: createMainWindow()     ← 最后才开窗口
```

这个顺序不是随便排的，它体现了正确的启动逻辑：

顺序	做什么	为什么必须先做
1	注册 IPC 通道	窗口一开，界面马上就会发请求，通道必须先就位
2	打开数据库	界面要显示账目，数据必须先准备好
3	创建窗口	前两步就绪了，才能让用户看到界面

如果顺序反了（先开窗口再连数据库），界面会在数据没准备好时就去请求，用户会看到一闪而过的错误或空白。

这就是"读代码"的第一个实用技巧：看**顺序**，顺序本身就在表达设计意图。

步骤 4：感受一下"改一个功能要动几个文件"

不用真改，只是**找出来**。任务是：**在侧栏加一个新的导航项，叫「我的页面」**。

在项目里搜一下，你会需要碰：

```
# 1. 侧栏导航列表（界面显示）
Select-String -Path src\renderer\src\layouts\AppLayout.tsx -Pattern "NAV" | Select-Object

# 2. 路由表（决定点进去显示什么）
Select-String -Path src\renderer\src\App.tsx -Pattern "Route path" | Select-Object -First
```

结论：一个"加个页面"的需求，至少要动**两个文件**（导航 + 路由），还要新建一个页面组件。

这个练习的目的：让你对"改动的影响范围"有概念。这是模块 4 的核心能力——**动手前先想清楚要碰哪些文件**。

卡住了怎么办

现象	原因	怎么办	
两个数字都是 0	通配符没生效（PowerShell 版本差异）	换成 `Get-ChildItem src\main -Recurse -Filter *.ts \	Select-String "from 'node:"`
找不到 `NAV`	变量名可能不同	先 `Get-Content src\renderer\src\layouts\AppLayout.tsx -TotalCount 40` 看开头	
看不懂 `webPreferences`	正常，这是 Electron 的配置对象	只要记住四条的含义即可，不必理解全部	

四、自测题

1. 主进程和渲染进程分别负责什么？用餐厅比喻说一下。

2. 为什么界面不能直接读数据库？说出至少两个理由。

3. `nodeIntegration: false` 在防什么？如果改成 `true` 会有什么风险？

4. 项目里 `src\shared\` 是干什么的？为什么需要它？

5. 启动顺序是"注册通道 → 打开数据库 → 创建窗口"。如果反过来先创建窗口，会出什么问题？

答案与解析

1. 主进程 = 后厨：管数据、读写文件、调系统。**渲染进程 = 服务员**：管界面、响应用户。中间只通过 **preload（传菜口）** 和 **IPC（点菜单）** 沟通。 → 口诀：后厨管数据，服务员管笑脸，中间只走传菜口。

2. ① **安全**：界面代码不可信，若它有文件权限，一段恶意脚本就能删掉账本；② **稳定**：界面崩溃不连累数据和主进程；③ **分工**：两边技术栈不同（Node vs 浏览器），混在一起会打架。 → 任一理由都成立，但**安全**是根本原因。

3. 它在防"界面代码直接使用系统能力"（比如 `require('fs')` 读写文件）。 → 改成 `true` 后，**界面就能直接读写你的文件，包括账本。**所以这个值必须是 `false`，没有正当理由改它。

4. 放**两边都要用的东西**：类型定义、常量、纯计算函数。 → 因为后厨和服务员必须对"一笔账有哪些字段""金额怎么算"有**完全一致的理解**，否则口径会错。放在 `shared\` 就能保证一致。

5. 窗口一出现，界面立刻就会向主进程请求数据（显示账目、账户列表等），但此时**通道还没注册、数据库还没打开**，请求会失败，用户看到空白或报错。 → 所以启动顺序体现的是"依赖关系"：**先准备好能力，再开窗口给用户用。**

五、术语表

术语	一句话解释
主进程 / main	应用的后厨，管数据、文件、系统，是唯一能碰数据库的进程
渲染进程 / renderer	服务员，负责画界面、响应用户操作，**碰不到系统**
preload	主进程与渲染进程之间唯一的合法通道
IPC	进程间通信机制，"点菜单"，界面请求后厨干活的正式途径
`contextIsolation`	隔离界面代码与通道代码，防止互相篡改
`nodeIntegration`	是否允许界面直接使用 Node 能力；**必须为 false**
`webSecurity`	浏览器安全策略开关，应为 true
`sandbox`	最严格的沙箱模式
`shared\`	主进程与渲染进程共用的类型、常量与纯计算
架构 / Architecture	软件各部分的划分与协作方式
玻璃房 / 沙箱	被限制权限、只能通过指定通道与外界交互的运行环境
技术债	为了当下能跑通而做的妥协，将来要还的账

六、本讲与项目的对应

- 你查看的文件：`src\main\index.ts`（启动顺序）、`src\main\window.ts`（安全配置）、`src\renderer\src\layouts\AppLayout.tsx`
- 你验证的事实：渲染层 0 处系统导入 / 主进程十几处
- **本讲不修改任何代码，不影响你的记账数据**
- 上一讲：1-3 代码变成软件 ｜ 下一讲：**2-2 传菜口：preload 与 IPC**（把这条链路彻底走通）

讲义版本：v1 ｜ 对应项目版本：0.1.0